A. Java Fundamentals (🟢 Questions 1–20)

1. What is Java?

Java is a general-purpose programming language used to build applications such as desktop programs, websites, Android apps, and server-side software. It is strongly typed, which means variables have declared types and Java checks how values are used.

Java source code is usually saved in a file ending in .java. The Java compiler converts it into **bytecode**, which is then run by the **Java Virtual Machine (JVM)**. Because JVMs are available for different operating systems, the same compiled Java program can often run on Windows, macOS, or Linux without being rewritten.

For example, a Java program that calculates a bill can run as a small command-line application. The same language is also used to build larger systems with databases, web services, and user interfaces.

2. What are the main features of Java?

Java has several useful features:

- **Object-oriented:** Programs are commonly organized around classes and objects.
- **Platform-independent:** Java bytecode can run on any compatible JVM.
- **Strongly typed:** Java checks that values are used according to their declared types.
- **Automatic memory management:** The garbage collector can reclaim memory used by objects that are no longer needed.
- **Exception handling:** Programs can respond to many runtime problems in a structured way.
- **Multithreading support:** Java includes tools for running multiple tasks concurrently.
- **Large standard library:** It includes ready-made tools for common tasks such as handling files, collections, dates, and networking.

These features help developers build both small programs and large applications. They do not automatically make a program correct or secure; good design and careful coding are still important.

3. Why is Java called platform independent?

Java is called platform-independent because the compiler generally does not translate source code directly into instructions for just one operating system or processor. Instead, it produces **bytecode**, which a JVM can run.

The process is:

1. A programmer writes Java source code.
2. The Java compiler converts the source into bytecode.
3. A JVM on a particular computer loads and runs that bytecode.

For example, a class file compiled on one operating system can often run on another if that system has a compatible JVM. This is the idea behind the phrase **“write once, run anywhere.”**

Portability is not absolute: a program that uses operating-system-specific features or native libraries may need changes to run elsewhere.

4. What is the difference between source code, bytecode, and machine code?

These are different forms of a program at different stages:

Form	What it is	Example of how it is used
Source code	Human-readable instructions written by the programmer. Java source files usually end in .java.	A programmer writes System.out.println("Hello");.
Bytecode	Intermediate instructions produced by the Java compiler, usually stored in .class files.	The JVM loads and executes the bytecode.
Machine code	Instructions for a specific processor.	A computer’s CPU executes machine code directly.

The Java compiler turns source code into bytecode. The JVM then interprets the bytecode or compiles it into machine code for the current computer.

5. What is the JVM?

The **Java Virtual Machine**, or **JVM**, is the runtime software that executes Java bytecode. It acts as a layer between a Java program and the computer’s operating system and hardware.

The JVM performs several tasks, including:

- Loading Java classes.
- Checking bytecode for certain errors and safety issues.
- Running bytecode by interpreting it or compiling it into machine code.
- Managing memory used by objects.
- Running garbage collection to reclaim memory that is no longer needed.

Different operating systems use different JVM implementations. Those JVMs can run compatible Java bytecode, which helps Java programs work across platforms.

6. What is the JDK?

The **Java Development Kit**, or **JDK**, is a collection of tools for developing Java programs. It includes the Java compiler and other development tools, along with the components needed to run Java programs.

For example, the javac command compiles a source file:

javac Hello.java

The JDK is intended for developers who need to write, compile, debug, or package Java applications. Build tools such as Maven or Gradle can use the JDK to automate these tasks for larger projects.

7. What is the JRE?

The **Java Runtime Environment**, or **JRE**, provides the components needed to run Java applications. Conceptually, it includes the JVM and the runtime libraries that Java programs depend on.

A person who only needs to run a Java application needs a compatible runtime. A developer generally installs a JDK, which also provides tools such as the compiler. Java packaging has changed over time, so the JRE may not always be distributed as a separate installation.

8. What is the difference between JDK, JRE, and JVM?

The terms describe related but different parts of Java:

Component	Main purpose
JVM	Executes Java bytecode and handles runtime tasks such as memory management.
JRE	Provides the JVM and libraries needed to run Java applications.
JDK	Provides development tools, such as the compiler, along with runtime components.

A simple way to remember the difference is: **the JVM runs bytecode, the JRE provides the runtime, and the JDK provides tools for developing Java software.**

9. What is bytecode?

Bytecode is the intermediate code produced when the Java compiler compiles a .java source file. It is usually stored in a .class file.

Bytecode is not ordinary Java source code, and it is not machine code for one particular processor. It is designed for the JVM, which checks and executes it. This lets different JVM implementations run compatible bytecode on different computer platforms.

For example, compiling Hello.java typically creates Hello.class, which contains bytecode for the Hello class.

10. Why is Java considered an object-oriented programming language?

Java is considered object-oriented because most Java programs are organized using **classes** and **objects**:

- A **class** defines the fields and methods that its objects can have.
- An **object** is a particular instance of a class.
- **Encapsulation** groups data with related behavior and controls access to that data.
- **Inheritance** allows a class to extend another class.

- **Polymorphism** allows different objects to respond to the same method call in different ways.
- **Abstraction** exposes important behavior while hiding some implementation details.

For example, a Vehicle class might define a move() method. Car and Bicycle classes could provide different implementations. Java also has primitive values such as int and boolean, so not every value in Java is an object.

11. What is the role of the main() method in Java?

For a conventional Java application launched by the Java launcher, main() is the entry point where the application starts. The launcher calls the method, and the statements inside it begin executing.

A common example is:

```
public class Hello {
  public static void main(String[] args) {
    System.out.println("Hello, Java!");
  }
}
```

Here, the program prints a message when it starts. The args parameter can contain values supplied on the command line when launching the application.

Other environments, such as test frameworks or application servers, may start Java code through a different mechanism.

12. Why is main() declared as public static void?

In the traditional Java entry-point declaration, each keyword has a purpose:

- **public** means the launcher can access the method.
- **static** means the launcher can call it without first creating an object of the class.
- **void** means the method does not return a value.
- **String[] args** provides an array for command-line arguments.

The commonly taught form is:

```
public static void main(String[] args)
```

Some modern Java versions and launch modes support additional entry-point forms, but this remains the standard form used in most introductory examples.

13. Can we execute a Java program without a main() method?

A regular class-based application launched directly by the Java launcher normally needs an accepted main() entry point. If the launcher cannot find one, it does not know where to start the application.

Java code can still run without its own main() method when another program or tool calls it. For example, a test framework can call test methods, an application server can call application components, and JShell can execute code interactively.

So the answer depends on how the code is being run: a directly launched application generally needs an entry point, while a library or framework component may not.

14. What is a Java class?

A class is a definition for a Java type. It describes the fields, constructors, and methods that objects of that type can have.

For example:

```
class Car {
   String color;
    void drive() {
      System.out.println("The car is moving.");
   }
}
```

This class defines a color field and a drive() method. It does not represent one specific car by itself; it describes what a Car object can contain and do.

15. What is an object?

An object is a particular instance of a class created while a program is running. It has its own identity and state, and it can use the behavior defined by its class.

For example:

```
Car myCar = new Car();
myCar.color = "blue";
myCar.drive();
```

Here, new Car() creates an object. The variable myCar holds a reference to it, and the object's color field is set to "blue". Another Car object could have a different color.

16. What is the difference between a class and an object?

A **class** is a definition that describes a type. An **object** is a specific instance created from that class.

For example, Car can be a class. A blue car and a red car can be two different objects created from it. Both objects have the fields and methods defined by the class, but each object can have its own field values.

A class is written in the program; objects are created and used while the program runs.

17. What is a reference variable?

A reference variable holds a reference to an object or array. It does not contain the object's complete contents directly.

For example:

Car myCar = new Car();

The myCar variable refers to the Car object created by new Car(). Assigning myCar to another reference variable copies the reference, so both variables may refer to the same object.

A reference can also have the value null, meaning it does not currently refer to an object. Trying to use null as though it referred to an object can cause a NullPointerException.

18. What is the difference between primitive data types and reference data types?

Primitive types hold simple values directly. Reference types hold references to objects or arrays.

Feature	**Primitive type**	**Reference type**
Examples	int, double, char, boolean	String, arrays, and objects made from classes
What the variable holds	A simple value	A reference to an object or array
Can it be null?	No	Yes
Equality with ==	Compares the values	Checks whether references point to the same object

For example:

int age = 20;

String name = "Ravi";

age holds an integer value. name refers to a String object. Java also provides wrapper classes such as Integer when an integer needs to be treated as an object.

19. What are the eight primitive data types in Java?

Java has eight primitive types:

Type	**General purpose**
byte	Small whole numbers
short	Whole numbers with a larger range than byte
int	Commonly used whole numbers
long	Whole numbers with a larger range than int
float	Floating-point numbers with single precision

Type	General purpose
double	Floating-point numbers with double precision
char	A single UTF-16 code unit, written with single quotes
boolean	The values true or false

For example:

```
int count = 5;

double price = 12.50;

char grade = 'A';

boolean isReady = true;
```

For calculations requiring exact decimal values, such as financial amounts, BigDecimal is often more suitable than float or double.

20. What is type casting?

Type casting converts a value from one type to another. Some conversions happen automatically, while others require an explicit cast.

A **widening conversion** moves to a type that can generally represent a larger range of values:

```
int number = 25;

double value = number;
```

A **narrowing conversion** may lose information, so it usually requires an explicit cast:

```
double price = 12.75;

int wholeNumber = (int) price;
```

In the example, wholeNumber becomes 12 because the fractional part is discarded. Casting a value does not guarantee it will fit safely, so range and precision should be considered.

B. Variables, Operators and Control Statements (🟢 Questions 21–36)

21. What is a local variable?

A **local variable** is declared inside a method, constructor, or block of code. It can be used only within the part of the program where it is declared.

```
void showTotal() {
   int total = 25;  // local variable
   System.out.println(total);
}
```

Here, total exists only inside showTotal(). It cannot be used by another method unless its value is passed to that method or returned from this one.

Unlike fields, local variables do **not** receive automatic default values. Java requires you to assign a value before using one.

22. What is an instance variable?

An **instance variable** is a field declared inside a class but outside its methods and without the static keyword. Each object created from the class has its own copy of the variable.

```
class Student {
   String name;  // instance variable
   int age;     // instance variable
}
```

If you create two Student objects, each can have a different name and age. Instance variables describe the individual state of an object. Java gives them default values when the object is created—for example, 0 for an int and null for a reference.

23. What is a static variable?

A **static variable** is declared with the static keyword. It belongs to the class rather than to individual objects, so the class's instances share it.

```
class Student {
   static String school = "Central School";
   String name;
}
```

Every Student object can have its own name, but all share the same school value. A static variable can be accessed through the class name, such as Student.school.

Static variables are useful for values shared across instances. Constants are often declared using both static and final, such as static final double TAX_RATE = 0.08;.

24. What is the difference between == and =?

The single equals sign (=) **assigns** a value to a variable. The double equals sign (==) **compares** two values or references and produces true or false.

```
int score = 90;      // assigns 90 to score
boolean passed = score == 90;  // compares score with 90
```

In the example, score = 90 changes the variable's value. The expression score == 90 checks whether the value is 90.

For primitive values, == compares the values. For object references, it checks whether the references point to the same object; see Question 25 for that distinction.

25. What is the difference between == and .equals()?

For **primitive values**, == compares the values. For **object references**, == checks whether both references point to the exact same object. The .equals() method checks logical equality as defined by the object's class.

```
String first = new String("Java");

String second = new String("Java");

System.out.println(first == second);      // false: different objects

System.out.println(first.equals(second));  // true: same text
```

The String class defines .equals() to compare the characters in two strings. Other classes can define their own meaning of equality.

Use == when you specifically want to check object identity. Use .equals() when you want to check whether two objects represent equal values.

26. What are relational operators in Java?

Relational operators compare values and produce a boolean result: true or false.

Java's comparison operators are:

- < — less than
- > — greater than
- <= — less than or equal to
- >= — greater than or equal to
- == — equal to
- != — not equal to

For example:

```
int age = 20;

if (age >= 18) {
   System.out.println("Adult");
}
```

Here, age >= 18 evaluates to true, so the message is printed. Relational operators are commonly used in if statements and loop conditions. They work with compatible primitive values; for comparing object values, methods such as .equals() or a comparator are generally needed.

27. What are logical operators?

Logical operators combine or invert boolean expressions. Java's main logical operators are:

- && — logical AND
- || — logical OR
- ! — logical NOT

```
int age = 20;

boolean hasId = true;

if (age >= 18 && hasId) {

  System.out.println("Entry allowed");

}
```

The condition is true only if both parts are true. With ||, the result is true if at least one part is true. The ! operator reverses a boolean value: !true is false.

Java also has & and |, which can operate on booleans without short-circuiting and can perform bitwise operations on integers.

28. What is the difference between && and &?

With boolean values, both operators represent AND, but they evaluate their operands differently:

- && is **short-circuit AND**. If the first condition is false, Java skips the second because the whole result must already be false.
- & evaluates **both** boolean operands, even when the first is false.

```
if (age >= 18 && hasPermission()) {

  // hasPermission() runs only if age >= 18 is true

}
```

Short-circuiting can improve efficiency and can prevent unsafe expressions from running. For example, code can check that an object is not null before calling one of its methods:

```
if (name != null && name.length() > 0) {

  // Safe to check name.length()

}
```

A single & also performs a bitwise AND when used with integer values.

29. What is the ternary operator?

The **ternary operator**, also called the conditional operator, is a short way to choose between two expressions. Its form is:

```
condition ? valueIfTrue : valueIfFalse
```

Example:

```
int age = 20;

String category = age >= 18 ? "Adult" : "Minor";
```

If the condition is true, category receives "Adult"; otherwise, it receives "Minor". The operator is called ternary because it uses three parts: a condition and two possible results.

It is most readable for simple choices. For several actions or complex conditions, an if-else statement is usually clearer.

30. What is the difference between break and continue?

break and continue change how a loop runs:

- **break** immediately exits the nearest enclosing loop.
- **continue** skips the remaining statements in the current iteration and moves to the next iteration.

```
for (int i = 0; i < 5; i++) {
  if (i == 2) {
    continue; // skips printing 2
  }
  System.out.println(i);
}
```

This loop prints 0, 1, 3, and 4. If the continue were replaced with break, the loop would stop entirely when i reached 2.

break can also exit a switch statement. continue applies to loops.

31. What is the difference between while and do-while?

A while loop checks its condition **before** running its body. If the condition is false initially, the body runs zero times.

```
while (condition) {
  // repeated statements
}
```

A do-while loop checks its condition **after** running its body, so the body runs at least once.

```
do {
  // repeated statements
} while (condition);
```

A do-while loop is useful when an action must happen before deciding whether to repeat—for example, displaying a menu before checking whether the user wants to continue. Remember the semicolon after the while condition in a do-while loop.

32. When would you use a for loop?

Use a for loop when you want to repeat instructions and can describe the loop with an **initial value**, a **continuation condition**, and an **update**.

```
for (int i = 0; i < 5; i++) {
```

```
    System.out.println(i);
}
```

This loop starts i at zero, runs while i < 5, and adds one to i after each iteration. A for loop is especially useful for counting or visiting array elements by index:

```
for (int i = 0; i < names.length; i++)
 System.out.println(names[i]);
}
```

When you need to visit every item but do not need its index, an enhanced for loop can be simpler:

```
for (String name : names) {
    System.out.println(name);
}
```

33. What is a nested loop?

A **nested loop** is a loop placed inside another loop. The inner loop runs during each iteration of the outer loop.

```
for (int row = 0; row < 3; row++) {
    for (int column = 0; column < 2; column++) {
        System.out.println(row + ", " + column);
    }
}
```

The outer loop runs three times, and the inner loop runs twice for each outer iteration, so the body runs six times in total. Nested loops are useful for processing rows and columns, such as a grid or table.

The total amount of work can grow quickly. If both loops run n times, the inner body runs about n × n times.

34. What is a switch statement?

A switch statement selects one branch of code based on the value of an expression. It can be clearer than a long chain of if-else statements when checking one value against several specific options.

```
int day = 2;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
```

```
  case 2:
    System.out.println("Tuesday");
    break;
  default:
   System.out.println("Another day");
}
```

Here, day is compared with the case values. Since it is 2, the program prints "Tuesday". In traditional colon-style syntax, break prevents execution from falling into the next case. Modern Java also supports arrow-style cases, which do not fall through.

35. Can a switch statement work with a String?

Yes. Java supports String values in a switch statement.

```
String command = "start";
switch (command) {
  case "start":
    System.out.println("Starting");
    break;
  case "stop":
    System.out.println("Stopping");
    break;
  default:
    System.out.println("Unknown command");
}
```

String matching is case-sensitive, so "start" and "Start" are different values. If input should be treated without regard to case, normalize it first—for example, by converting it to a consistent case. A switch on a null string will throw a NullPointerException, so handle possible null values before the switch.

36. What happens if break is omitted from a switch case?

In a traditional colon-style switch, leaving out break usually causes **fall-through**: execution continues into the statements of the following case, even if that case label does not match.

```
int choice = 1;
switch (choice) {
  case 1:
    System.out.println("One");
```

```
case 2:
    System.out.println("Two");
    break;
```

C. Arrays and Strings (/ Questions 37–55)

37. What is an array?

An **array** is an object that stores a fixed number of elements, all of the same type. Each element has an index, starting at 0.

```
int[] scores = {80, 92, 75};
System.out.println(scores[0]); // prints 80
```

Here, scores holds three integers. The first element is at index 0, the second at index 1, and the third at index 2. Arrays are useful when you want to store several related values and access them by position.

38. Why are arrays called fixed-size data structures?

An array's length is set when it is created and cannot be changed afterward.

```
int[] numbers = new int[3]; // length is 3
```

You can change the values stored in those three slots, but you cannot add a fourth slot to that same array. You could create a larger array and copy the values into it, but the original array remains length 3. Use an ArrayList or another resizable collection when the number of items may change.

39. How do you declare and initialize an array in Java?

You can create an array by listing its values:

```
int[] ages = {18, 21, 25};
```

Or declare a length and let Java initialize the elements with default values:

```
int[] ages = new int[3];
```

The second array contains three integers, initially all 0. You can assign values by index:

```
ages[0] = 18;
ages[1] = 21;
ages[2] = 25;
```

The usual declaration style is type[] name, such as String[] names.

40. What is the default value of elements in an integer array?

Every element in a newly created int array defaults to 0.

```
int[] values = new int[3];
```

```
System.out.println(values[0]); // prints 0
```

Java initializes array elements automatically. For example, boolean elements default to false, and reference-type elements—such as String[]—default to null. This differs from local variables inside methods, which must be assigned before you use them.

41. How do you find the length of an array?

Use the array's length field, without parentheses:

```
int[] values = {10, 20, 30};

System.out.println(values.length); // prints 3
```

This is a field, not a method. That differs from a String, where you call text.length() with parentheses. A common loop uses i < values.length as its condition so it stops before reaching an invalid index.

42. What happens when you access an invalid array index?

Java throws an ArrayIndexOutOfBoundsException if you use an index below 0 or greater than or equal to the array's length.

```
int[] values = {10, 20, 30};

System.out.println(values[3]); // throws ArrayIndexOutOfBoundsException
```

The valid indexes here are 0, 1, and 2. A safe loop condition is i < values.length, not i <= values.length. The exception happens at runtime, so the code may compile even though the index is invalid.

43. What is a two-dimensional array?

A **two-dimensional array** is an array whose elements are themselves arrays. It is often used to represent rows and columns, such as a grid.

```
int[][] grid = {
  {1, 2, 3},
  {4, 5, 6}
};

System.out.println(grid[1][2]); // prints 6
```

The first index selects the row; the second selects an element in that row. Java implements this as an array of arrays, so rows can have different lengths. Such an arrangement is often called a *jagged array*.

44. What is the difference between an array and an ArrayList?

Feature	Array	ArrayList
Size	Fixed when created	Can grow or shrink
Length	Use the length field	Use the size() method

Feature	Array	ArrayList
Stored values	Can store primitives or objects	Stores objects; primitive values are boxed
Common operations	Read or assign by index	Methods such as add(), remove(), and contains()

For example, an int[] can directly store primitive int values, while an ArrayList<Integer> stores Integer objects. Java's autoboxing usually handles conversions between int and Integer automatically.

Choose an array when the number of elements is fixed or you need direct, simple indexed storage. Choose an ArrayList when you need to add or remove elements conveniently.

45. What is a String in Java?

A String is a Java class that represents a sequence of characters. String literals are written inside double quotation marks:

String message = "Hello, Java!";

Strings provide methods for tasks such as comparing text, finding a substring, checking whether text contains another sequence, and changing letter case. Strings are **immutable**, which means their contents cannot be changed after they are created; methods that appear to change a string return a new string.

46. Why is String immutable?

A String is immutable because once it is created, its character contents cannot be changed. For example, a concatenation creates a new string instead of altering the original:

String word = "Java";

word.concat(" programming");

System.out.println(word); // still prints Java

To keep the result, assign it:

word = word.concat(" programming");

Immutability makes strings safe to share across parts of a program, helps keep their hash codes stable when used as map keys, and supports Java's string pool. For lots of repeated modifications, use StringBuilder.

47. What is the difference between String, StringBuilder, and StringBuffer?

Type	Mutable?	Typical use
String	No	Text that is created and then used without repeated modification
StringBuilder	Yes	Building or changing text repeatedly, usually within one thread
StringBuffer	Yes	Mutable text when synchronized methods are specifically needed

Because String is immutable, repeated concatenation can create many temporary strings. StringBuilder lets you append to one changeable sequence:

```
StringBuilder text = new StringBuilder();

text.append("Hello");

text.append(" Java");

String result = text.toString();
```

StringBuffer has synchronized methods, but that does not automatically make every operation around it thread-safe. StringBuilder is generally the simpler choice unless synchronized access is needed.

48. What is the difference between equals() and equalsIgnoreCase()?

Both methods compare the contents of two strings. The difference is whether letter case matters:

```
String first = "Java";

String second = "JAVA";

System.out.println(first.equals(second));         // false

System.out.println(first.equalsIgnoreCase(second)); // true
```

Use equals() when uppercase and lowercase letters should be considered different. Use equalsIgnoreCase() when case should not matter, such as matching a simple command. For locale-sensitive language comparisons, Java provides more specialized comparison tools.

49. What is the use of length() in a String?

String.length() returns the number of UTF-16 code units in a string:

```
String text = "Hello";

System.out.println(text.length()); // prints 5
```

The result can be used to check whether a string is empty or to control a loop over its positions. Be aware that some Unicode symbols use two UTF-16 code units, so length() does not always equal the number of visible symbols. Use codePointCount() when you need a count of Unicode code points.

50. What is the use of substring()?

substring() returns part of a string. The two-index form includes the start index and excludes the end index:

```
String text = "Programming";

String part = text.substring(0, 7);

System.out.println(part); // prints Program
```

The one-index form returns from that index to the end:

String ending = text.substring(7);

The original string is not changed because strings are immutable. The indexes must be within valid bounds, or Java throws an exception.

51. What is the use of charAt()?

charAt(index) returns the char at a specified position in a string. String indexes begin at 0.

String word = "Java";

char first = word.charAt(0);

System.out.println(first); // prints J

The index must be at least 0 and less than the string's length. charAt() works well for simple character-by-character processing. Some Unicode symbols occupy two char positions, so use code-point methods when processing text that may include those symbols.

52. What is the difference between trim() and strip()?

Both methods return a string with whitespace removed from its beginning and end. They do not remove whitespace from the middle.

- trim() uses a limited, older definition of whitespace.
- strip() uses Unicode-aware whitespace rules and is available from Java 11 onward.

String text = " hello ";

System.out.println(text.trim()); // "hello"

System.out.println(text.strip()); // "hello"

For ordinary spaces, both often give the same result. strip() is generally better when you need Unicode-aware whitespace handling.

53. What do toUpperCase() and toLowerCase() do?

These methods return a new string with letters converted to uppercase or lowercase. They do not modify the original string.

String word = "Java";

String upper = word.toUpperCase(); // "JAVA"

String lower = word.toLowerCase(); // "java"

Case conversion may depend on the computer's language locale. For identifiers or other text that should be handled consistently regardless of locale, use toUpperCase(Locale.ROOT) or toLowerCase(Locale.ROOT).

54. What is the use of contains() in String?

contains() checks whether a string includes a particular character sequence. It returns true if the sequence is present and false otherwise.

String sentence = "Java is useful";

System.out.println(sentence.contains("useful")); // true

The check is case-sensitive: "Java".contains("java") returns false. The argument is treated as plain text, not as a regular expression. Use startsWith() or endsWith() when you specifically need to check the beginning or ending of a string.

55. What is the difference between concat() and the + operator for strings?

Both can join strings and produce a result:

String greeting = "Hello".concat(" Java");

String message = "Hello" + " Java";

The + operator is more flexible because it can combine strings with other values by converting them to text:

int count = 3;

String message = "Count: " + count;

concat() takes another String. Neither method changes the original string. For many repeated additions—especially inside a loop—use StringBuilder to avoid creating many intermediate strings.

D. OOP Concepts (🟡 Questions 56–80)

56. What are the four main principles of OOP?

The four commonly taught principles of **object-oriented programming (OOP)** are:

1. **Encapsulation:** Groups data with the methods that work on it, and controls how that data can be accessed.
2. **Inheritance:** Lets a class build on another class and reuse or specialize its behavior.
3. **Polymorphism:** Lets code use a shared type or method call while different objects provide different behavior.
4. **Abstraction:** Shows the important features of a type while hiding some implementation details.

For example, a Vehicle type might define a general move() operation. A Car and a Bicycle could provide different implementations of that operation. These principles help organize code, though they are design tools rather than rules that every class must use.

57. What is encapsulation?

Encapsulation means keeping an object's data and the operations that use that data together, while controlling how other code can access or change it.

For example, a BankAccount class might store its balance privately and provide a deposit() method. The method can reject negative deposits and protect the account from invalid changes.

Encapsulation helps a class maintain valid state and makes its implementation easier to change. Other code interacts with the class through its public behavior instead of depending on its internal details.

58. How is encapsulation achieved in Java?

A common way to achieve encapsulation is to declare fields private and provide public methods for the operations callers should be allowed to perform.

```
class BankAccount {
   private double balance;
   public double getBalance() {
      return balance;
   }
  public void deposit(double amount) {
      if (amount > 0) {
         balance += amount;
      }
   }
}
```

Here, code outside BankAccount cannot directly assign a new value to balance. It must use the methods provided by the class, and deposit() validates the amount.

Encapsulation does not mean adding a getter and setter for every field. The class should expose meaningful operations that preserve its rules.

59. What is inheritance?

Inheritance lets one class extend another class. The new class, called a **subclass**, inherits accessible behavior from the existing **superclass** and can add or specialize its own behavior.

```
class Animal {
   void eat() {
      System.out.println("Eating");
   }
}
class Dog extends Animal {
   void bark() {
      System.out.println("Barking");
   }
```

}

A Dog object can use both eat() and bark(). Java supports single class inheritance: a class can directly extend only one class, though it can implement multiple interfaces.

60. Why is inheritance used?

Inheritance is used to model a genuine **"is-a"** relationship, share behavior among related classes, and enable polymorphism.

For example, if Dog extends Animal, code that works with an Animal can also work with a Dog. The Dog class can reuse general animal behavior and add dog-specific behavior.

Inheritance should be used when the subclass truly can be used wherever its superclass is expected. If one class simply needs to use another class's capabilities, **composition**—having one object contain or use another—may be a more flexible choice.

61. What is the difference between extends and implements?

Keyword	Used for	Meaning
extends	A class or an interface	A class extends one superclass. An interface can extend one or more interfaces.
implements	A class	The class promises to provide the behavior required by one or more interfaces.

Example:

```
class Dog extends Animal implements Trainable, Pet {
  // Implement required interface methods here
}
```

Here, Dog inherits from the Animal class and implements two interfaces. If a class does not implement every required interface method, it must be declared abstract.

62. What is polymorphism?

Polymorphism means that code can work with a common type while different actual objects provide different behavior.

```
Animal pet = new Dog();
pet.makeSound();
```

If Dog overrides makeSound(), Java runs the Dog implementation even though the variable is declared as Animal. This lets a program handle different kinds of objects through a shared superclass or interface.

Polymorphism makes programs easier to extend: new classes can provide their own behavior while still fitting into code written for the common type.

63. What is compile-time polymorphism?

The term **compile-time polymorphism** usually refers to **method overloading**. A class has multiple methods with the same name but different parameter lists, and the compiler chooses which one matches the call.

```
void print(int number) {
   System.out.println(number);
}
void print(String text) {
   System.out.println(text);
}
```

Calling print(5) selects the int version; calling print("Hello") selects the String version. The compiler uses the argument types and the method declarations to make its choice.

64. What is runtime polymorphism?

Runtime polymorphism occurs when Java chooses an overridden instance method based on the actual object's class at runtime.

```
Animal pet = new Dog();
pet.makeSound();
```

If Dog overrides makeSound(), Java calls that version. The reference variable's declared type is Animal, but the object itself is a Dog.

Runtime polymorphism is useful when code should work with a general type—such as an interface—while each implementation supplies its own behavior.

65. What is method overloading?

Method overloading means defining multiple methods with the same name but different parameter lists. The lists can differ in the number, types, or order of parameters.

```
int add(int a, int b) {
   return a + b;
}
double add(double a, double b) {
   return a + b;
}
```

The compiler chooses the method that best matches the arguments. The return type alone cannot distinguish overloaded methods. Overloading can make an API convenient when related operations accept different kinds of input.

66. What is method overriding?

Method overriding occurs when a subclass provides a compatible implementation of an inherited instance method.

```
class Animal {
  void makeSound() {
    System.out.println("Some sound");
  }
}
class Dog extends Animal {
  @Override
  void makeSound() {
    System.out.println("Bark");
  }
}
```

When makeSound() is called on a Dog object, the Dog implementation runs. The @Override annotation is recommended because the compiler checks that the method really overrides an inherited method.

67. What is the difference between method overloading and overriding?

Feature	Overloading	Overriding
Definition	Same method name with different parameter lists	Subclass provides a compatible implementation of an inherited instance method
Where it occurs	Can occur in one class or across inheritance	Requires an inherited method
Method selection	Compiler chooses based on argument types and context	Runtime chooses based on the actual object
Return type	Return type alone cannot distinguish overloads	Must be compatible with the inherited method's return type

Overloading provides different ways to call a similarly named operation. Overriding lets a subclass customize inherited behavior.

68. Can we overload a method by changing only its return type?

No. Java does not allow two methods to have the same name and the same parameter list if they differ only in return type.

```
int getValue() { return 1; }
// Not allowed in the same class:
```

```
double getValue() { return 1.0; }
```

The compiler uses the method name and parameters to determine which method a call refers to. Since the return type is not enough to make the call unambiguous, the parameter list must differ for an overload.

69. What is the super keyword?

super refers to the superclass part of the current object. It is commonly used to:

- Call a superclass constructor with super(...).
- Call an overridden superclass method, such as super.display().
- Refer to a superclass field when a subclass has a field with the same name.

```
class Dog extends Animal {
  @Override
  void makeSound() {
    super.makeSound();
    System.out.println("Bark");
  }
}
```

An explicit super(...) constructor call must be the first statement in the subclass constructor. If no constructor call is written, Java attempts to call the superclass's no-argument constructor.

70. What is the this keyword?

this refers to the current object. It is often used when a constructor parameter has the same name as a field:

```
class Student {
  private String name;
    Student(String name) {
    this.name = name;
  }
}
```

Here, this.name means the field belonging to the current object, while name means the constructor parameter.

this(...) can also call another constructor in the same class. That constructor call must be the first statement in the constructor. A static method cannot use this because it is not associated with a particular object.

71. What is a constructor?

A **constructor** is a special class member that initializes an object when it is created. It has the same name as the class and does not declare a return type.

```
class Student {
  String name;
    Student(String studentName) {
    name = studentName;
  }
}
Student student = new Student("Maya");
```

The constructor runs as part of new Student("Maya") and sets the new object's name.

If a class declares no constructors, Java may provide a default no-argument constructor. Once you declare a constructor yourself, Java does not automatically provide that default constructor.

72. What is the difference between a constructor and a method?

Feature	Constructor	Method
Purpose	Initializes a new object	Performs an operation or provides behavior
Name	Must match the class name	Can have any valid method name
Return type	Has no return type	Declares a return type, including void
How it runs	Runs when an object is created	Called by program code when needed

A constructor sets up an object's initial state. A method can read or change that state or perform some other task after the object has been created.

73. Can a constructor be overloaded?

Yes. A class can have multiple constructors with different parameter lists. This lets callers create objects in different ways.

```
class Student {
  String name;
  int age;
    Student(String name) {
     this.name = name;
  }
    Student(String name, int age) {
     this.name = name;
     this.age = age;
```

```
  }
}
```

The compiler selects a constructor based on the arguments provided with new. One constructor can call another constructor in the same class using this(...), which can reduce repeated initialization code.

74. Can a constructor be inherited?

No. Constructors are not inherited by subclasses. A subclass must declare its own constructors, though those constructors can call a superclass constructor using super(...).

```
class Animal {
  Animal(String name) { }
}
class Dog extends Animal {
  Dog(String name) {
    super(name);
  }
}
```

Here, Dog does not inherit the Animal constructor. Its constructor explicitly calls the superclass constructor to initialize the inherited part of the object.

If the superclass has no accessible no-argument constructor, a subclass constructor must call an appropriate superclass constructor explicitly.

75. What is an abstract class?

An **abstract class** is a class declared with the abstract keyword. It cannot be instantiated directly, but other classes can extend it.

An abstract class can contain:

- Abstract methods, which declare behavior without providing a method body.
- Concrete methods, which already have an implementation.
- Fields and constructors.

```
abstract class Shape {
  abstract double area();
    void describe() {
    System.out.println("This is a shape.");
  }
}
```

A concrete subclass must implement the inherited abstract methods before it can be instantiated. Abstract classes are useful when related classes share state or common behavior but the general base type should not be created directly.

76. What is an interface?

An **interface** is a Java reference type that describes a contract—behavior that implementing classes agree to provide.

```
interface Printable {
  void print();
}
class Report implements Printable {
  public void print() {
    System.out.println("Printing report");
  }
}
```

A class that implements Printable must provide its required abstract methods unless the class is abstract. Modern Java interfaces can also contain default, static, and private methods. Interface fields are constants, not ordinary per-object instance fields.

77. Why do we use interfaces?

Interfaces let code depend on a **contract** rather than one particular implementation. Different classes can implement the same interface and be used through the interface type.

For example, several classes might implement a PaymentMethod interface. A checkout program can call the shared payment operation without needing to know each implementation's internal details.

Interfaces also make implementations easier to replace and can help with testing. A program can use a simplified test implementation of an interface instead of depending on an external service.

78. What is the difference between an abstract class and an interface?

Feature	Abstract class	Interface
Main purpose	Provides a shared base class, often with common state or implementation	Defines a contract or capability that different classes can implement
Fields	Can have instance fields	Cannot have ordinary per-instance fields; fields are constants
Constructors	Can have constructors	Cannot have constructors

Feature	Abstract class	Interface
Methods	Can have abstract and implemented instance methods	Can have abstract methods and, in modern Java, default, static, and private methods
Inheritance	A class can extend only one class	A class can implement multiple interfaces

Use an abstract class when closely related classes need to share state or substantial implementation. Use an interface when different types should promise the same behavior, even if they do not share a class hierarchy. A class can extend one class and implement multiple interfaces.

79. Can a class implement multiple interfaces?

Yes. A Java class can implement more than one interface by listing them after implements, separated by commas.

```
class SmartPrinter implements Printable, Scannable, Faxable {
  // Implement the required methods
}
```

The class must provide implementations for the required abstract methods unless it is declared abstract. If two interfaces provide conflicting default methods with the same signature, the class must resolve the conflict by providing its own implementation or explicitly choosing behavior.

80. Can an interface extend another interface?

Yes. An interface can extend one or more other interfaces.

```
interface Readable {
  void read();
}
interface Editable extends Readable {
  void edit();
}
```

Editable includes the contract from Readable and adds edit(). A concrete class that implements Editable must provide implementations for the required methods from both interfaces, unless it is abstract. An interface can extend multiple interfaces, separated by commas.

E. Access Modifiers, Packages & Core Concepts (🔴 Questions 81–90)

81. What are the access modifiers in Java?

Access modifiers control where classes, fields, constructors, and methods can be accessed from. Java has four access levels:

- **private** — accessible only within the declaring class.
- **Package-private** — accessible within the same package. This is the default when no access modifier is written.
- **protected** — accessible within the same package and also in qualifying subclass contexts.
- **public** — accessible wherever the declaring type is accessible.

For example, a class might make its internal fields private while exposing a few operations as public. That allows other code to use the class without directly changing its internal state.

Top-level classes can generally be declared public or package-private. Nested classes can use all four access levels.

82. What is the difference between private, default, protected, and public?

Access level	Where it can be accessed
private	Within the declaring class, including its nested code under Java's access rules.
Default (package-private)	From classes in the same package. No keyword is written.
protected	From classes in the same package and from subclasses in qualifying contexts.
public	Wherever the declaring type itself is accessible.

Example:

```
class Example {
   private int secret;
   int packageValue;       // package-private
   protected int familyValue;
   public int openValue;
}
```

protected needs particular care across packages: it does not simply mean “public to every subclass.” Access from a subclass in another package is subject to Java’s protected-access rules.

83. Why are data members generally declared private?

Fields are generally declared private to protect an object’s state. If outside code could freely assign any value, it might put the object into an invalid or inconsistent state.

Instead, the class can provide methods that validate changes:

```
class BankAccount {
```

```
    private double balance;

    public void deposit(double amount) {

        if (amount > 0) {

            balance += amount;

        }

    }

}
```

Here, code outside the class cannot directly assign to balance. It must use deposit(), which rejects non-positive amounts. Private fields also make it easier to change how a class stores its data without changing the code that uses its public methods.

84. What is a package?

A **package** is a namespace that groups related Java types, such as classes and interfaces. A source file can declare its package near the beginning:

```
package com.example.school;

public class Student {

}
```

In a conventional project, the package name corresponds to the folder structure. For example, com.example.school is commonly stored in folders named com/example/school.

Packages help organize code and avoid naming conflicts. Two packages can contain classes with the same short name, while their fully qualified names remain distinct.

85. Why are packages used in Java?

Packages are used to:

1. **Organize related types:** For example, a program might group its model, user-interface, and database classes into different packages.
2. **Avoid naming conflicts:** Two packages can each contain a class named Date, identified by different fully qualified names.
3. **Control access:** Package-private members are accessible to code in the same package but not generally to unrelated packages.

Package names are usually lowercase. Organizations often use a reversed domain name, such as com.example.app, to make package names distinctive.

86. What is the difference between import and package?

A **package declaration** says which package a source file's types belong to. An **import declaration** lets the source file refer to another package's types by their shorter names.

```
package com.example.app;
```

```
import java.util.ArrayList;

public class Main {

  ArrayList<String> names = new ArrayList<>();

}
```

Here, package com.example.app; places Main in that package. The import lets the code write ArrayList rather than the full name java.util.ArrayList.

An import does not copy code into the file and does not grant access that the access modifiers forbid. You can always use a type's fully qualified name instead of importing it.

87. What is the purpose of the static keyword?

The static keyword marks a member as belonging to the **class**, rather than to each individual object.

A static field is shared by instances:

```
class Student {

  static String school = "Central School";

  String name;

}
```

All Student objects share school, while each object has its own name. A static method can be called using the class name, such as Math.max(3, 7).

A static method does not have a this reference, because it is not called on one particular object. It therefore cannot directly use instance fields or methods without an object reference. Static is also used with nested classes and initializer blocks.

88. What is the purpose of the final keyword?

The meaning of final depends on what it modifies:

- A **final variable** can be assigned only once.
- A **final method** cannot be overridden by a subclass.
- A **final class** cannot be subclassed.

Example:

```
final int MAX_USERS = 100;
```

The variable MAX_USERS cannot be assigned a different value after it has been initialized. A common convention is to write names for constants in uppercase.

For a reference variable, final prevents the variable from pointing to a different object, but it does not necessarily make that object's contents immutable.

89. Can a final variable be modified?

A final variable cannot be **reassigned** after it has been initialized. What that means depends on whether it holds a primitive value or an object reference.

```
final int count = 5;

// count = 6; // Not allowed
```

For a reference, the reference cannot be changed to point to another object, but the referenced object may still be mutable:

```
final ArrayList<String> names = new ArrayList<>();

names.add("Asha"); // Allowed: the list changes

// names = new ArrayList<>(); // Not allowed: the reference changes
```

So final fixes the variable's value or reference; it does not automatically make a referenced object immutable.

90. Can a final method be overridden?

No. A subclass cannot override a method declared final in its superclass.

```
class Parent {
  final void showMessage() {
    System.out.println("Hello");
  }
}

class Child extends Parent {
  // Cannot override showMessage()
}
```

A class may declare a final method when subclasses should not change that method's behavior. However, a subclass can still define a same-named method with a **different parameter list**; that is overloading, not overriding.

F. Exception Handling (🔴 Questions 91–100)

91. What is an exception?

An **exception** is an object that represents an unusual condition or problem that occurs while a program is running. When an exception occurs, Java interrupts the normal sequence of statements and looks for code that can handle it.

For example, reading a missing file may cause an IOException. Trying to access an array with an invalid index may cause an ArrayIndexOutOfBoundsException.

Exceptions belong to Java's Throwable family. Most application code deals with exceptions rather than directly creating or handling objects of the more serious Error type.

92. What is exception handling?

Exception handling is the process of detecting and responding to exceptional conditions so a program can fail in a controlled way or recover when possible.

Java provides several keywords for this:

- try encloses code that might throw an exception.
- catch handles a matching exception.
- finally contains cleanup code that normally runs as control leaves the try/catch.
- throw raises an exception.
- throws declares that a method may pass exceptions to its caller.

For example, a program might catch a file-related exception and tell the user that the file could not be opened. If the current method cannot respond appropriately, it can let the exception propagate to a method that has more context.

93. What is the difference between an error and an exception?

Both Error and Exception are subclasses of Java's Throwable class, but they usually represent different kinds of problems.

Type	Typical meaning	Usual application response
Exception	A condition an application may be able to anticipate or handle, such as an unavailable file.	Catch it when there is a useful response, or declare it so a caller can handle it.
Error	A serious problem involving the runtime or environment, such as running out of memory.	Ordinary application code generally should not try to recover from it.

This is a general guideline, not an absolute rule. For instance, OutOfMemoryError is an Error, and catching it usually cannot make the program's state safe again.

94. What is the difference between checked and unchecked exceptions?

The key difference is whether the compiler requires the program to catch or declare the exception.

Kind	Compiler requirement	Examples
Checked exception	Must be caught with catch or declared with throws.	IOException, SQLException
Unchecked exception	No catch-or-declare requirement.	NullPointerException, IllegalArgumentException

Checked exceptions are subclasses of Exception that are not subclasses of RuntimeException. Unchecked exceptions include subclasses of RuntimeException; Java also treats Error types as unchecked.

A method that reads a file may declare throws IOException, requiring its callers to consider that failure. A NullPointerException does not need to be declared, though the program should still prevent or appropriately address its cause.

95. What is the purpose of try and catch?

A try block encloses code that may throw an exception. A catch block handles an exception of a specified type if one occurs in the associated try.

```
try {
   int result = 10 / divisor;
   System.out.println(result);
} catch (ArithmeticException e) {
   System.out.println("The divisor cannot be zero.");
}
```

If divisor is zero, Java throws ArithmeticException, and the matching catch block runs. If no exception occurs, the catch block is skipped.

Catch an exception when your code can respond meaningfully—for example, by asking for different input or displaying a useful message. Catching an exception and doing nothing with it hides the problem and makes debugging harder.

96. What is the purpose of the finally block?

A finally block is used for cleanup that should normally happen whether the try block completes successfully or an exception occurs.

```
try {
   System.out.println("Working with a resource");
} catch (Exception e) {
   System.out.println("An operation failed");
} finally {
   System.out.println("Cleanup");
}
```

The finally block normally runs as control leaves the associated try/catch, including when the try block returns or throws an exception. It is commonly used to release resources.

For resources that implement AutoCloseable, such as files or streams, **try-with-resources** is usually preferred:

```
try (var reader = new java.io.BufferedReader(
      new java.io.FileReader("notes.txt"))) {
   System.out.println(reader.readLine());
```

}

Java closes the resource automatically. A finally block is not guaranteed to run if the JVM stops abruptly, such as when the process is forcibly terminated.

97. What is the difference between throw and throws?

throw and throws are related, but they are used in different places:

Keyword	Where it appears	Purpose
throw	In a method or other block of code	Actually raises a particular exception object.
throws	In a method or constructor declaration	Declares exceptions that may pass from the method to its caller.

Example:

```
void checkAge(int age) {
if (age < 0) {
    throw new IllegalArgumentException("Age cannot be negative");
  }
}
void readFile() throws IOException {
  // File-reading code that may cause an IOException
}
```

The first method uses throw to raise an exception. The second uses throws to declare that an IOException may propagate to the caller.

98. Can we create our own exception class?

Yes. A program can define a custom exception class to represent a failure specific to its problem domain.

Extend Exception to create a **checked exception**:

```
class InsufficientFundsException extends Exception {
  InsufficientFundsException(String message) {
    super(message);
  }
}
```

Or extend RuntimeException to create an **unchecked exception**:

```
class InvalidOrderException extends RuntimeException {
```

```
  InvalidOrderException(String message) {
    super(message);
  }
}
```

A custom exception can give callers a clear way to recognize and handle a particular condition. Use one when it communicates a meaningful domain-specific problem; otherwise, a standard Java exception may be more appropriate.

99. What happens if an exception is not handled?

If an exception is not caught in the method where it occurs, it **propagates up the call stack**. Java looks for a matching catch block in the calling methods.

If no code handles it before it reaches the top of the current thread, the exception is **uncaught**. The thread terminates, and the runtime typically reports the exception and a stack trace showing where it occurred. Other threads in the application may continue running.

An uncaught exception does not always shut down the entire Java process; the outcome depends on whether other threads are still active and how the application is structured.

100. Why should exceptions be handled instead of allowing the program to terminate unexpectedly?

Handling exceptions can make failures more understandable and keep a program's behavior controlled. Depending on the situation, code may:

- Display a useful message instead of a confusing runtime trace.
- Ask the user to correct invalid input.
- Retry an operation that may succeed later.
- Choose a reasonable fallback.
- Close resources such as files or network connections.
- Report the failure to an appropriate part of the application.